

LIBRARY CATALOGUE

Introduction

The Library Catalogue System is a RESTful web service designed to manage information about books, authors, categories, and borrowing records in a digital library environment. The system allows librarians and authorized users to perform operations such as creating, updating, searching, and deleting catalogue data through standardized HTTP requests.

The API is intended to simplify library management by providing centralized access to catalogue resources and supporting efficient organization of library materials. Users can search books by title, author, category, or publication year, as well as monitor the availability of copies and borrowing activity.

The system follows REST architectural principles and uses JSON as the primary data exchange format. The API supports filtering, pagination, authentication, standardized error handling, and caching mechanisms to ensure scalability, usability, and performance.

The main entities of the system are:

- Books
- Authors
- Categories
- Borrow Records
- Users

The API is designed according to the Richardson Maturity Model and uses appropriate HTTP methods and status codes for all operations.

Entities

Book

The Book entity represents a library item available in the catalogue. It stores general information about books, including title, author, category, publication details, and availability status.

Fields

- id — unique identifier of the book
- title — title of the book

- isbn — International Standard Book Number
- publicationYear — year of publication
- description — short description of the book
- availableCopies — number of available copies
- totalCopies — total number of copies in the library
- authorId — reference to the author
- categoryId — reference to the category
- createdAt — creation timestamp
- updatedAt — last update timestamp

Author

The Author entity stores information about book authors.

Fields

- id — unique identifier of the author
- firstName — author's first name
- lastName — author's last name
- birthDate — date of birth
- nationality — nationality of the author
- biography — short biography
- createdAt — creation timestamp
- updatedAt — last update timestamp

Category

The Category entity is used to organize books into different genres or thematic groups.

Fields

- id — unique identifier of the category
- name — category name
- description — category description
- createdAt — creation timestamp
- updatedAt — last update timestamp

User

The User entity represents a system user who can borrow books and access protected API functionality.

Fields

- id — unique identifier of the user
- firstName — user's first name
- lastName — user's last name
- email — user email address
- passwordHash — encrypted user password
- role — user role (ADMIN, LIBRARIAN, MEMBER)
- createdAt — creation timestamp
- updatedAt — last update timestamp

Borrow Record

The Borrow Record entity stores information about borrowing operations in the library system.

Fields

- id — unique identifier of the borrow record
- bookId — identifier of the borrowed book
- userId — identifier of the user who borrowed the book
- borrowDate — date when the book was borrowed
- dueDate — deadline for returning the book
- returnDate — actual return date
- status — borrowing status (BORROWED, RETURNED, OVERDUE)
- fineAmount — penalty amount for overdue return
- createdAt — creation timestamp
- updatedAt — last update timestamp

Functional Requirements

The Library Catalogue System must support the following functionality:

- Users must be able to view the list of books available in the catalogue.
- Users must be able to search books by title, author, category, ISBN, and publication year.

- The system must support filtering and sorting of books.
- Authorized users must be able to create new books.
- Authorized users must be able to update existing book information.
- Authorized users must be able to delete books from the catalogue.
- Users must be able to view detailed information about authors and categories.
- The system must support book borrowing and returning operations.
- The system must track book availability and borrowing history.
- The system must automatically detect overdue borrow records.
- The API must support pagination for all collection endpoints.
- The system must provide authentication and role-based authorization.

Non-functional Requirements

The system must satisfy the following non-functional requirements:

- The API must follow REST architectural principles.
- All data must be exchanged in JSON format.
- The API must use HTTPS for secure communication.
- The system must be stateless.
- API response time should not exceed 2 seconds under normal load.
- The system must support caching for frequently requested resources.
- The API must provide meaningful HTTP status codes.
- The system must ensure data consistency and validation.
- The API must support scalability and maintainability.
- Authentication must be implemented using JWT tokens.
- The system must include standardized error responses.

REST API Design

Books

GET /books

Returns a paginated list of books available in the library catalogue.

Query Parameters

Parameter	Description
page	Page number
pageSize	Number of items per page
title	Filter by book title
author	Filter by author
category	Filter by category
year	Filter by publication year
sort	Sort results

Example Request

GET /books?page=1&pageSize=10&category=Fantasy

Possible Status Codes

Code	Description
200	Request completed successfully
400	Invalid query parameters
401	Unauthorized
500	Internal server error

GET /books/{id}

Returns detailed information about a specific book.

Possible Status Codes

Code	Description
200	Book retrieved successfully
404	Book not found
401	Unauthorized
500	Internal server error

POST /books

Creates a new book in the catalogue.

Request Body

```
{  
  "title": "Clean Code",  
  "isbn": "9780132350884",  
  "publicationYear": 2008,  
  "availableCopies": 5,  
  "totalCopies": 5,  
  "authorId": 3,  
  "categoryId": 2  
}
```

Possible Status Codes

Code	Description
201	Book created successfully
400	Invalid request data
401	Unauthorized
403	Forbidden
409	Book already exists
500	Internal server error

PUT /books/{id}

Updates information about an existing book.

Possible Status Codes

Code	Description
200	Book updated successfully
400	Invalid request data
401	Unauthorized
403	Forbidden
404	Book not found

Code	Description
500	Internal server error

DELETE /books/{id}

Deletes a book from the catalogue.

Possible Status Codes

Code	Description
204	Book deleted successfully
401	Unauthorized
403	Forbidden
404	Book not found
500	Internal server error

Author

GET /authors

Returns a paginated list of authors.

Query Parameters

- page
- pageSize
- name

Status Codes

Code	Description
200	Authors retrieved successfully
400	Invalid query parameters
500	Internal server error

GET /authors/{id}

Returns detailed information about a specific author.

Status Codes

Code	Description
200	Author retrieved successfully
404	Author not found
500	Internal server error

POST /authors

Creates a new author.

Status Codes

Code	Description
201	Author created successfully
400	Invalid request data
401	Unauthorized
403	Forbidden

PUT /authors/{id}

Updates author information.

DELETE /authors/{id}

Deletes an author.

Categories

GET /categories

Returns all book categories

GET /categories/{id}

Returns category details.

POST /categories

Creates a new category.

PUT /categories/{id}

Updates category information.

DELETE /categories/{id}

Deletes a category.

Users

POST /auth/login

Authenticates a user and returns a JWT token.

Request Body

```
{  
  "email": "user@example.com",  
  "password": "password123"  
}
```

Status Codes

Code	Description
200	Login successful
401	Invalid credentials
500	Internal server error

GET /users/{id}

Returns user information.

POST /users

Creates a new user account.

Borrow Records

GET /borrow-records

Returns borrowing history records.

Query Parameters

- page
- pageSize
- userId
- status

POST /borrow-records

Creates a new borrowing operation.

Request Body

```
{  
  "bookId": 4,  
  "userId": 12,  
  "borrowDate": "2026-05-22",  
  "dueDate": "2026-06-05"  
}
```

Status Codes

Code	Description
201	Borrow record created successfully
400	Invalid request data
404	Book or user not found
409	No available copies
500	Internal server error

PUT /borrow-records/{id}/return

Marks a borrowed book as returned.

Status Codes

Code	Description
200	Book returned successfully
404	Borrow record not found
409	Book already returned

Filtering and Pagination

The API supports filtering, sorting, and pagination for all collection endpoints in order to improve performance and usability.

Pagination

Pagination is implemented using query parameters:

- page — page number
- pageSize — number of items per page

Example Request

GET /books?page=1&pageSize=10

Example Response

```
{
  "page": 1,
  "pageSize": 10,
  "totalItems": 120,
  "totalPages": 12,
  "data": [
    {
      "id": 1,
      "title": "Clean Code"
    }
  ]
}
```

Filtering

The API supports filtering using query parameters.

Examples

GET /books?title=Harry Potter

GET /books?authorId=5

GET /books?categoryId=2

GET /borrow-records?status=OVERDUE

Sorting

Sorting is supported using the sort query parameter.

Example

GET /books?sort=title

Authentication

The API uses JWT (JSON Web Token) authentication to protect secured endpoints.

Users must authenticate using the login endpoint and include the received token in the Authorization header of protected requests.

Authentication Header

Authorization: Bearer <jwt-token>

Login Endpoint

POST /auth/login

Example Request Body

```
{  
  "email": "user@example.com",  
  "password": "password123"  
}
```

Example Response

```
{  
  "token": "jwt-token"  
}
```

Authorization Roles

The system supports role-based access control:

- ADMIN — full access to all resources
- LIBRARIAN — management of books and borrow records
- MEMBER — access to book searching and borrowing functionality

Error Handling

The API provides standardized error responses in JSON format.

Example Error Response

```
{  
  "timestamp": "2026-05-22T14:00:00Z",  
  "status": 404,  
  "error": "Not Found",  
  "message": "Book not found"  
}
```

Common Status Codes

Code	Description
200	Successful request
201	Resource created successfully
204	Resource deleted successfully
400	Invalid request
401	Unauthorized
403	Forbidden
404	Resource not found
409	Conflict
500	Internal server error

Caching

The API supports caching for frequently requested resources in order to improve performance and reduce server load.

Caching is primarily applied to GET requests.

The API uses the Cache-Control HTTP header to define caching policies.

Example Header

Cache-Control: public, max-age=300

This configuration allows clients to cache responses for 5 minutes.

Resources such as books, authors, and categories may be cached because they do not change frequently.

Sensitive or user-specific data must not be cached.

Richardson Maturity Model

The designed API follows the Richardson Maturity Model principles.

Level 1 — Resources

The API uses separate resources for different entities:

/books

/authors

/categories

/users

/borrow-records

Level 2 — HTTP Methods

The API uses appropriate HTTP methods for operations:

Method	Purpose
GET	Retrieve resources
POST	Create resources
PUT	Update resources
DELETE	Remove resources

Level 3 — HATEOAS

The API may include hyperlinks to related resources in responses.

Example Response

```
{
  "id": 1,
  "title": "Clean Code",
  "links": [
    {
      "rel": "author",
      "href": "/authors/3"
    },
    {
      "rel": "category",
      "href": "/categories/2"
    }
  ]
}
```

